

SVTR: from image to phone

A visual engineering handbook for scene text recognition.

SVTR

UNDERSTAND THE FEATURES.
TRACE THE TEXT.

B O _ O K

B O _ O K -> BOOK

The blank separates the two O characters. Page 9 explains why.

Your goal: make useful changes in a phone OCR team

You need to connect architecture, code, model inputs and device behavior. A bad result might come from cropping, CTC alignment, a symbol dictionary, numerical precision or runtime placement. Learn to isolate the cause and propose a measurable fix.

What you will build up to

Start with pixels and attention, walk through original SVTR and CTC, then study SVTRv2 and its variants. Finish with code navigation, evaluation, export, quantization, profiling and a concrete engineering lab. The examples use simple English with short Hinglish memory hooks.

Read -> predict -> inspect -> reproduce

This is a companion to hands-on experiments. It gives you the map; practical competence comes from running models and debugging real crops. No Samsung internals or measured flagship-phone performance is assumed. SVTR is the intended model name behind 'svrt'.

Sources: [P1](#) [P2](#)

Your clickable learning map

Use the chapter tiles, bookmarks and Back / Next buttons.

01 / FOUNDATIONS

Pages 3-6

OCR system, tensors, overlapping patches, attention

02 / ORIGINAL SVTR

Pages 7-11

Mixing, hierarchy, CTC, loss and T/S/B/L variants

03 / SVTRv2

Pages 12-15

What changed: MSR, FRM and training-only guidance

04 / PHONE ENGINEERING

Pages 16-20

Code, evaluation, export, precision and profiling

05 / CHECK & PRACTISE

Pages 21-22

Answers, diagnosis, lab checklist and primary sources

Tap to check your reasoning

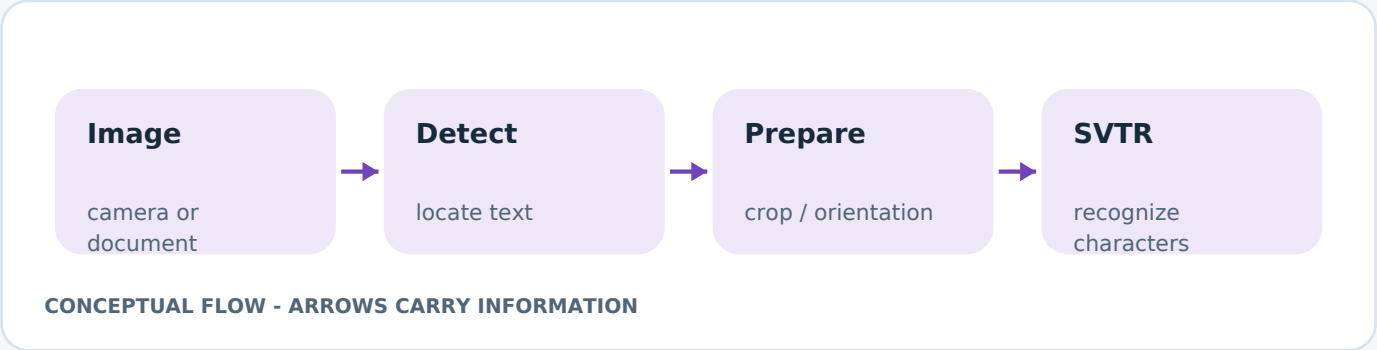
Gold **TRY** strips ask you to predict an outcome. Tap **Answer** to jump to page 21, then tap the answer number to return. The final page has five fillable checkboxes for your first lab. A full PDF reader generally supports more interactions than an embedded preview.

Three evidence labels

Verified configuration: exact behavior in the named official code/config. **Teaching example:** invented numbers or simplified diagrams. **Engineering proposal:** an experiment to validate on your device. Paper results retain their benchmark context. Sources and runtime guidance checked on 8 October 2026.

A recognizer inside an OCR system

SVTR reads a text region; the rest of the product still matters.



Detection and recognition solve different problems

A text **detector** returns regions. A text **recognizer** turns each region into a string. Layout logic then orders strings or assembles fields. SVTR does not by itself detect all page text, infer table structure or extract the correct invoice total. A page with 40 regions can require 40 recognizer inputs.

Why a single visual model is interesting

A convolutional neural network (**CNN**) learns local spatial filters. A recurrent neural network (**RNN**) processes sequences through a recurring state. Earlier recognizers often combined CNN + RNN + CTC. SVTR integrates local appearance and broad visual interactions into a hierarchical visual backbone, then uses a linear head and CTC decoding.

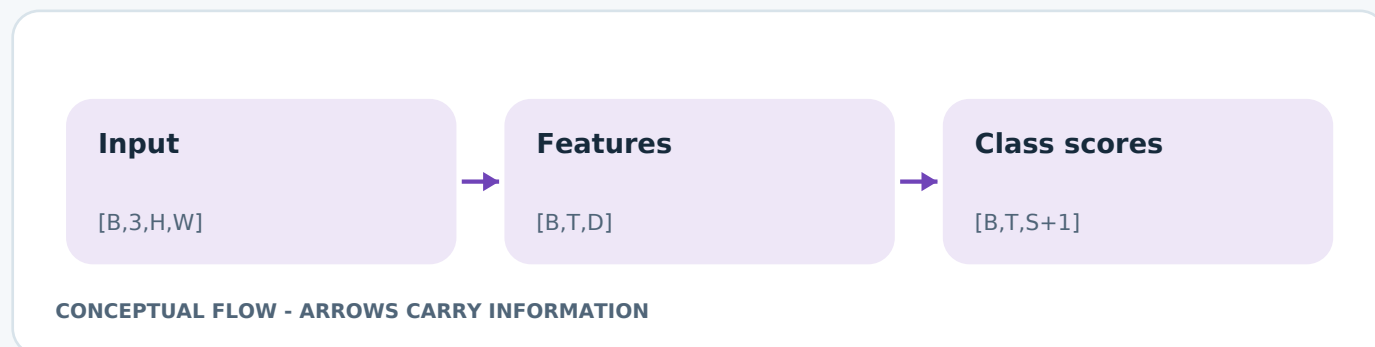
Approach	Where context comes from	Phone experiment
CRNN + CTC	Visual features plus recurrent sequence processing	Useful compact baseline on the same crops.
Original SVTR + CTC	Local/global visual mixing; no character feedback loop	Test speed, repeats, curves and long strings.
PARSeq	Image features plus permitted character embeddings in a decoder	Compare AR/NAR/refinement and end-to-end cost.

A precise meaning of visual-only

SVTR still learns associations across the whole word image. 'No separate sequence decoder' does not mean each pixel or letter is processed in isolation. CTC output factorization differs from a decoder conditioned explicitly on previous predicted characters.

Tensors, classes and the blank

A CTC time step is a feature position, not a video frame.



Read each dimension

A **tensor** is a shaped array of numbers. B is the number of crops in a batch. The 3 channels are image colour values; H and W are height and width. T is the number of feature positions submitted to the CTC classifier. D is feature width: the number of values in each position's vector. S is the number of ordinary symbols in the dictionary.

Weights, activations and embeddings

Weights / parameters are persistent learned values. **Activations** are intermediate values generated for this input. An **embedding** is a vector representing a patch or another object. D=64 means 64 numerical coordinates, not 64 recognised letters. A batch changes activations, not the trained parameter count.

Special item	CTC role	Common mistake
Blank	Predicts no emitted symbol at this step; enables alignment	Treating it as a visible space.
Space	An ordinary dictionary symbol if supported	Dropping spaces using blank-removal logic.
EOS / BOS	Not required by the basic CTC output scheme	Adding PARSeq stopping rules to CTC decoding.

An exact reference example

Original English SVTR uses 36 case-insensitive alphanumeric symbols plus blank: **37 output classes**. The reference Tiny setup has **25 CTC steps**, giving [B,25,37]. This is not a promise that every 25-character string is representable: adjacent repeated labels need extra steps.

TRY 01 Does a CTC blank mean the visible space in a sentence?

[Answer >](#)

Sources: [C2](#) [C4](#) [C5](#)

Patches with overlapping evidence

The stem progressively shrinks the image and expands feature channels.

TINY EXAMPLE: input to the SVTR backbone, after rectification



Effective stride 4; neighbouring tokens share receptive-field evidence.

What convolution and stride mean

A convolution applies a learned spatial filter across an image, combining nearby values into features. **Stride 2** moves the filter by two input locations per output location, reducing resolution. The original stem uses two 3 x 3 stride-2 convolutions with padding 1, each followed by BatchNorm and GELU. Tiny expands channels 3 -> 32 -> 64.

Overlap is different from disjoint cutting

Two such convolutions have a theoretical receptive field of **7 x 7** and effective stride **4**: first $r=3, j=2$; next $r=3+(3-1) \times 2=7, j=4$. Adjacent output positions are four pixels apart, so their evidence overlaps. An 'effective 4x downsampling' description does not mean the stem literally cuts disjoint 4 x 4 tiles.

From grid to token sequence

The 8 x 25 grid contains **200 tokens**. Flattening its spatial dimensions produces [B,200,64]. Original SVTR adds learned absolute position embeddings so the tokens retain location information. One token can contain part of a character, background or multiple strokes; it is not automatically one complete character.

Keep input stages separate

The PaddleOCR reference config resizes recognizer input to [3,64,256], then uses an STN rectification transform with a [32,100] backbone output. Do not send [64,256] directly to a backbone configured for [32,100]. Always name which boundary a shape describes.

TRY 02 After two stride-2 convolutions, is the effective stride 2 or 4?

[Answer >](#)

Sources: [C1](#) [C2](#)

Attention: compute a useful mixture

Q asks, K matches, V contributes information.

TOY TWO-SOURCE ATTENTION

Value A = [2,0]

weight 0.75

$0.75 \times [2,0] + 0.25 \times [0,4] = [1.5,1]$

Value B = [0,4]

weight 0.25

Query, key and value are arrays

Learned projections turn token vectors into **Q (query)**, **K (key)** and **V (value)**. Similarity between queries and keys produces scores. Softmax converts those scores into weights that sum to 1. Values are combined using the weights. These names are an analogy, not human-written search requests stored in the network.

```
scores = (Q @ K.T) / sqrt(d_head) + mask
weights = softmax(scores, axis=-1)
output = weights @ V
```

How the toy weights arise

Scores $\ln(3)$ and 0 become weights $3/(3+1)=0.75$ and $1/(3+1)=0.25$. A blocked source receives $-\infty$ before softmax and gets zero weight. In self-attention, Q, K and V are projections of the same token set. They are not usually identical arrays after projection.

A block includes more than attention

Multi-head attention uses several projection spaces, then combines their results. Tiny stage 1 has 64 dimensions and 2 heads: 32 dimensions per head. An **MLP** is a per-token feed-forward network. Residual connections add input to an update; LayerNorm normalizes features. GELU is a nonlinear activation, allowing more than a single linear mapping.

Normalization changes with layer type

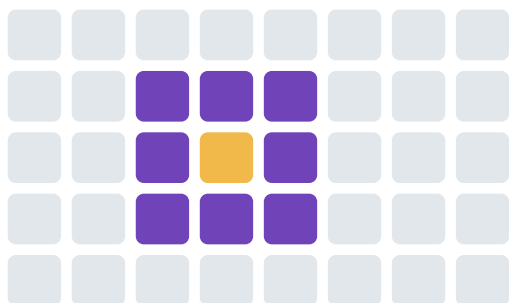
BatchNorm uses training statistics and stored evaluation statistics; LayerNorm normalizes each token's features. Evaluation mode also disables dropout. A converter must preserve the model's actual normalization order and epsilon; do not substitute a generic Transformer block.

Sources: [C1](#) [V1](#)

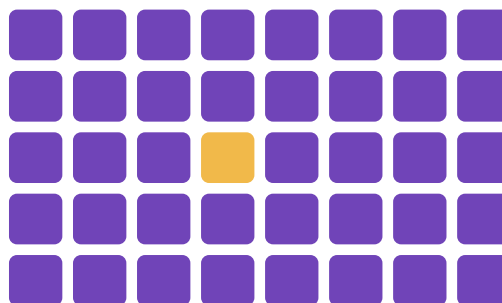
Local and global mixing

Early stroke evidence, then wider relationships between components.

LOCAL: nearby grid cells



GLOBAL: the full grid



Yellow = query. Coloured cells = eligible information sources.

Original SVTR local mixing uses attention

Each component attends within a sliding spatial neighbourhood; the reference window is 7 x 11. This can combine strokes and small character details. Global mixing lets a component attend to the full feature grid, connecting distant character parts and suppressing distracting background. Both are visual self-attention; neither is an AR character-generation loop.

A concrete implementation cost trap

In PaddleOCR's original Attention class, local mixing computes a **dense N x N score matrix**, then adds a local mask. Restricted information flow does not by itself produce sparse computation. An N=200, two-head FP32 score tensor has $2 \times 200 \times 200 = 80,000$ values, about **312.5 KiB**, excluding other buffers.

Mixer order is a separate meaning of permutation

Tiny uses six Local blocks followed by six Global blocks across its three stages. Small/Base/Large use other counts. Here 'permutation' means ordering the local/global block types. It is unrelated to PARSeq's character dependency permutations. Altering a mixer schedule changes the architecture and needs a compatible training experiment.

Debug the grid mask

Flattened indices must still correspond to the intended 2D neighbourhood. Check corners, row boundaries and mask shape on a tiny grid. A mask broadcast error can silently turn local attention into the wrong graph, or create an all-blocked row and NaNs.

TRY 03 Does adding a local mask guarantee sparse attention compute?

[Answer >](#)

Sources: [C1](#) [P1](#)

The full Tiny tensor route

Height shrinks; width positions survive until the CTC sequence.

Boundary: backbone input 32 x 100	Grid / sequence	Channels
After overlapping stem	8 x 25 = 200 tokens	64
Stage 1: 3 Local blocks	8 x 25	64
Merge 1; Stage 2: 3 Local + 3 Global	4 x 25 = 100 tokens	128
Merge 2; Stage 3: 3 Global	2 x 25 = 50 tokens	256
Combining: pool + projection	1 x 25	192
Reshape neck; linear CTC head	[B,25,37] scores	37 classes

Merging is intentionally asymmetric

A 3 x 3 convolution uses stride **[2,1]**: halve feature height while preserving feature width. LayerNorm follows. Increasing channel capacity compensates for some spatial compression. Keeping width protects the number of positions available for character alignment; shrinking width indiscriminately can make repeated or long labels impossible.

Combining makes a one-dimensional sequence

The reference last stage pools to [1,out_char_num], projects through a 1 x 1 convolution, then uses Hardswish and dropout. Here out_char_num=25, matching W/4. The reshape neck converts [B,192,1,25] into [B,25,192]. A linear classifier maps each feature position to ordinary-symbol and blank scores.

Paper shorthand versus code contract

The paper's combining sketch is height pooling to a width sequence. The Paddle implementation has **AdaptiveAvgPool2D([1,out_char_num])**, which can also fix sequence width. Increasing image width while keeping out_char_num=25 does not guarantee more CTC steps. Read the actual shape after pooling.

No character query is required here

These 25 positions are alignment slots, not 25 guaranteed letters. Several adjacent slots may represent one letter; some represent blank. CTC handles the many-to-one correspondence. Page 10 explains how training learns it without per-slot labels.

TRY 04 Does stride [2,1] halve width as well as height?

[Answer >](#)

Sources: [C1](#) [C2](#) [C4](#)

CTC: make BOOK stay BOOK

Collapse consecutive duplicate IDs, then remove blank IDs.

CORRECT COLLAPSE ORDER - _ denotes CTC blank

Path



Merge repeats

B O _ O K



Remove blanks

BOOK

If you remove _ first: B O O K collapses incorrectly to BOK.

The greedy path is not the final string

Take the highest-scoring class at each feature position to form a path. For `_ B B O _ O K K _`, first merge adjacent identical IDs to `_ B O _ O K _`, then remove blanks: **BOOK**. A blank between two equal letters keeps them distinct. This operation is often called the CTC collapse function.

A minimal correct decoder

The previous ID must track the **raw path**, including blanks. If you instead remember the previous emitted character, the second O after a blank can be mistakenly removed. Empty paths or all-blank paths should return an empty string under an explicit product policy.

```
previous = None
output = []
for token_id in path:
    if token_id != previous and token_id != blank_id:
        output.append(dictionary[token_id])
        previous = token_id
text = "".join(output)
```

Decode IDs before applying normalizations

The symbol dictionary defines the classes. Case folding or Unicode normalization before collapse can make different token IDs look equal and change repetition behavior. Decode with the checkpoint's dictionary first; apply any agreed text normalization as a separate stage.

TRY 05 What does the raw CTC path `O O _ O` decode to?

[Answer >](#)

Sources: [T1](#) [C5](#)

CTC loss learns the alignment

Training sums valid paths instead of assigning a letter to each slot.

TOY TARGET A, TWO STEPS, ALPHABET {A, _}

A _	0.18	collapses to A
_ A	0.28	collapses to A
A A	0.42	collapses to A

$$P(A) = 0.18 + 0.28 + 0.42 = 0.88$$

What the probabilities mean

At step 1 let $p(A)=0.6, p(\text{blank})=0.4$. At step 2 let $p(A)=0.7, p(\text{blank})=0.3$. The probability of a path is the product of its step probabilities. All three paths in the diagram collapse to A, so their probabilities are added. The only remaining path is blank-blank, with mass 0.12. CTC loss for target A is **$-\ln(0.88)$, about 0.128**.

Efficient training uses dynamic programming

CTC uses dynamic programming rather than enumerating exponentially many paths. Backpropagation computes gradients; the optimizer updates weights. Each visual step sees the image, but basic CTC factors per-step probabilities without previous output-character feedback.

Constraint	Worked consequence
Minimum steps	T must be at least U plus the number of adjacent repeated target pairs. BOOK: U=4, one OO pair, so $T \geq 5$.
Repeated labels	Target AAA needs at least A _ A _ A: five steps. Three slots cannot encode three consecutive As.
Invalid alignments	Bad input/target lengths can give infinite loss. zero_infinity masks such infinities; it does not make the sample learnable.

A training health check

Log invalid or too-short alignments, filtered labels, blank occupancy and validation edits. An all-blank model can look deceptively simple and fast. Verify dictionary IDs, blank ID, log-softmax axis and length tensors before adjusting the learning rate.

TRY 06 What is the minimum number of CTC steps for BOOK?

[Answer >](#)

Sources: [T1](#) [V5](#)

Original variants: T, S, B, L

Width, depth and mixer schedule change capacity and runtime.

Variant	Stage channels	Stage depths / heads	Local -> Global
Tiny	64 / 128 / 256	3 / 6 / 3; 2 / 4 / 8	6 -> 6
Small	96 / 192 / 256	3 / 6 / 6; 3 / 6 / 8	8 -> 7
Base	128 / 256 / 384	3 / 6 / 9; 4 / 8 / 12	8 -> 10
Large	192 / 256 / 512	3 / 9 / 9; 6 / 8 / 16	10 -> 11

Read a configuration by stage, not by nickname

Tiny and Small both use 192-dimensional combined features, while Base uses 256 and Large 384 in the paper. The block count and head count differ too. Dense projection weights often scale with channel width squared, but actual speed also depends on token counts, memory access and supported kernels. 'Tiny' is a hypothesis to benchmark, not a phone latency guarantee.

Paper backbone scope	Tiny	Small	Base	Large
Parameters, millions	4.15	8.45	22.66	38.81

Parameter-count scope matters

The paper's architecture table excludes rectification and the linear classifier. A full model/checkpoint can therefore have a different count. Multilingual dictionaries also change classifier size. Compare the same scope and precision; raw parameter count alone is not peak runtime memory.

SVTR in a neck is another architecture

PaddleOCR also implements **EncoderWithSVTR**, a compact module after a different backbone. It reduces channels, applies global mixing, and fuses the result with earlier features. A PP-OCR-style system using an SVTR neck is not automatically the standalone three-stage SVTR-T model. Inspect Backbone, Neck, Head and Transform together.

Record the complete identity

Model family/version + resolved config + checkpoint hash + dictionary + input transform + classifier + runtime defines your experiment. A supported mixer='Conv' implementation option does not imply the original paper's local attention and grouped convolution are interchangeable without retraining.

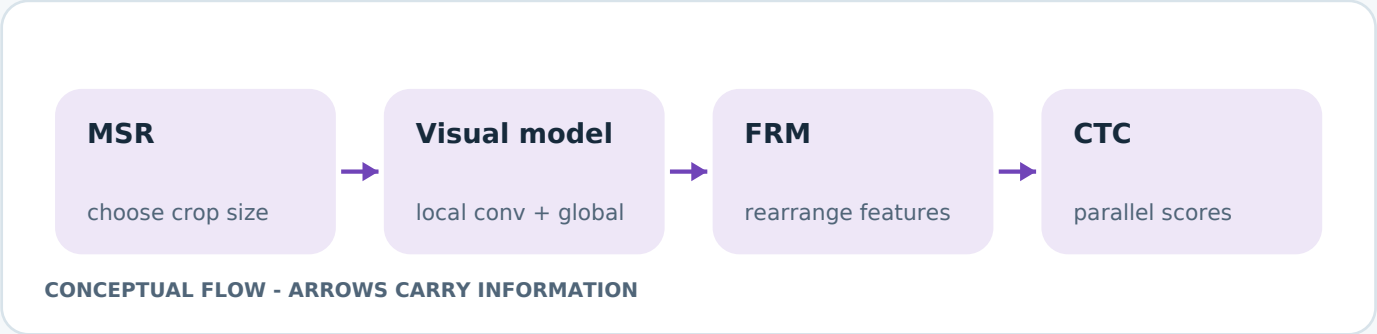
TRY 07 Is a CNN with an SVTR neck necessarily the original SVTR-T?

[Answer >](#)

Sources: [P1](#) [C1](#) [C2](#) [C3](#)

SVTRv2: the upgrades together

Better geometry, better alignment, stronger visual context learning.



Aspect	Original SVTR	SVTRv2 paper / named reference
Local mixing	Sliding neighbourhood attention	Two grouped 3x3 convolutions, with no activation/LN between them.
Late feature grid	H/16 x W/4	H/8 x W/4, retaining more vertical detail.
Position / geometry	Absolute positions; original config has TPS rectification	No absolute positions in published setup; MSR + FRM.
Training context	CTC objective	CTC plus semantic guidance; guidance removed for inference.

Separate three modules

MSR is Multi-Size Resizing: preserve readable text geometry through different input-size buckets. **FRM** is Feature Rearrangement Module: convert 2D features into a sequence suitable for CTC. **SGM** is Semantic Guidance Module: an auxiliary training path that uses surrounding strings to encourage richer visual features.

It remains a CTC recognizer at inference

SGM does not run as a spelling-correction loop on the phone. FRM remains in the inference path. The model outputs a sequence of symbol/blank distributions and applies CTC collapse. A filename containing 'Decoder' can implement feature selection and classification without autoregressive token generation.

Match the implementation, not a similar name

This guide uses OpenOCR's **SVTRv2LNConvTwo33** configs for the two-convolution reference. Other encoder files and later experiments can differ. Do not assume every class named SVTRv2 implements identical normalization, local kernels or export behavior.

MSR keeps the crop readable

Choose a size from aspect ratio; test each size as a runtime contract.

PAPER MSR BUCKETS - sizes shown as [height,width]



$R < 1.5$



1.5-2.5



2.5-3.5



$R \geq 3.5$

Ratio R = original W/H	Paper target H x W	Backbone step scale W/4
$R < 1.5$	64 x 64	16
$1.5 \leq R < 2.5$	48 x 96	24
$2.5 \leq R < 3.5$	40 x 112	28
$R \geq 3.5$	$32 \times (\text{floor}(R) \times 32)$	$8 \times \text{floor}(R)$

Calculate two actual choices

A 180 x 90 crop has $R=2.0$: choose **48 x 96**. A 590 x 100 crop has $R=5.9$: the paper rule selects **32 x 160**, giving 40 CTC positions after width downsampling. This is a bucket approximation; it does not preserve every aspect ratio exactly. Long-width caps and dataset samplers in an implementation still need inspection.

Geometry and deployment are coupled

A single square input can crush a long text line; a very wide input can distort a nearly vertical word. MSR reduces such forced distortion. On a phone, variable shapes can trigger graph recompilation, larger buffers or reduced accelerator support. A small set of precompiled width/height buckets is an engineering candidate to measure, not a change to apply blindly to a fixed-shape checkpoint.

Check the alignment capacity of every bucket

More width creates more sequence positions in the v2 path, but narrow buckets still have finite T. Count labels and repeated pairs after tokenization. Batch compatible sizes together; sorting only by word length is insufficient when image geometry differs.

TRY 08 For $R=5.9$, does the paper MSR rule choose width 160 or 189?

[Answer >](#)

Sources: [P2](#) [V3](#)

FRM arranges features for reading

Horizontal mixing, then learned vertical selection per output column.

TOY VERTICAL FEATURE SELECTION - one output column

value 2

weight 0.1

value 8

weight 0.7

value 4

weight 0.2

Selected = 6.6

$0.1 \times 2 + 0.7 \times 8 + 0.2 \times 4 = 6.6$; toy scalar values.

Why simply flattening can lose reading order

For curved or rotated text, character evidence may sit in different feature rows. Flattening all rows as a long raster sequence need not match the label's reading order. Averaging each column treats every row equally. FRM instead learns interactions across width and a content-dependent vertical mixture for each output column.

Follow RCTCDecoder in the reference code

Start with a 2D feature map $[B, D, h, w]$. A Transformer block processes each row's width sequence: $[B \times h, w, D]$. Learned projections create keys and values. A shared learned **char_token** scores locations; softmax runs along **height** separately for each column. Weighted values produce $[B, w, D]$, then a linear classifier gives $[B, w, S+1]$.

The token is not a predicted character

char_token is a learned selector shared across columns. It is not a BOS token, a known letter or one separate position query per final character. The module rearranges/selects visual information before CTC classification. The horizontal stage and encoder provide context that lets the vertical selector focus on useful evidence.

Keep three image operations distinct

Rectification changes pixels. **FRM** changes feature combinations. **CTC collapse** changes a symbol path into a string. Replacing one with another is an architecture change. FRM can improve alignment, but it cannot guarantee recovery of unreadable or completely missing visual content.

Sources: [V1](#) [V2](#)

SGM teaches, then steps aside

Context helps train visual features without a phone-side language loop.



A CAFE training example

When learning F, the left context can be **CA** and the right context **E**; F itself is excluded from those context strings. Each side is encoded into a context representation that attends to image features and predicts the target character. This auxiliary task encourages visual features to capture the context useful for locating and recognizing that character.

Two losses, different supervision

CTC learns sequence alignment from the word label. SGM uses character-level cross entropy for predictions from surrounding strings. The paper combines them as $L = 0.1 L_{ctc} + 1 L_{sgm}$, with left/right character predictions averaged inside the guidance loss. Those coefficients are the paper's setup, not universal fine-tuning defaults.

What changes between training and inference?

During training, labels and surrounding strings are available and gradients update the shared visual encoder. During inference, only the image is available: the SGM branch is omitted and its learned influence persists in encoder weights. The reference English recipe trains first without guidance, then with it. Check the exact checkpoint and phase before reproducing results.

Keep framework naming honest

OpenOCR's published configs expose a GTCDecoder wrapper with an SMTR guidance decoder and an RCTC branch; inference configs select the CTC path. That wrapper is not a reason to ship an AR decoder. Inspect inference flags and exported operators to confirm the actual graph.

TRY 09 Should the phone run SGM using the unknown ground-truth string?

[Answer >](#)

Sources: [P2](#) [V3](#) [V4](#)

Read the code with a baseline

Pin a configuration, checkpoint, dictionary and transform before changing them.

Reference	Look here	What to inspect
Original PaddleOCR	rec_svtrnet.py / rec_svtrnet.yml	PatchEmbed, Attention, SubSample, last_stage, STN and out_char_num.
SVTRv2 OpenOCR	svtrv2_inconv_two33.py	POPatchEmbed, local Conv blocks, sub_k, feat2d and use_pos_embed.
v2 feature-to-text path	rctc_decoder.py	Row attention, char_token, height softmax and final fc.
Training vs deploy	configs/rec/svtrv2/*.yml	Guidance wrapper, infer_gtc, dictionary, data resize and max_ratio.

SVTRv2 size configs you can identify

OpenOCR's Tiny RCTC setup uses channels [64,128,256], depths [3,6,3], heads [2,4,8]. Small uses [96,192,384], [3,6,3], [3,6,12]. Base uses [128,256,384], [6,6,6], [4,8,12]. These are v2 settings: original SVTR-S has a different final width and stage depth.

```
# Official OpenOCR inference pattern; install its pinned deps.
python tools/infer_rec.py \
  --c configs/rec/svtrv2/svtrv2_smtr_gtc_rctc_infer.yml \
  --o Global.infer_img=/path/to/crops
```

A useful first regression pack

Use clean text, blur, low contrast, narrow crop, repeated letters, case, punctuation, mixed scripts, curved text and a long SKU. Keep original crops, exact preprocessed tensors, per-step scores, argmax paths and decoded strings. Compare the official evaluation path before adding a custom mobile adapter. The command above is documented usage, not a run completed for this guide.

Logits or probabilities?

Both the inspected Paddle CTCHead and OpenOCR RCTCDecoder apply softmax in evaluation mode. Their training outputs are raw scores. Inspect the exported boundary before applying another softmax or log-softmax. Double softmax preserves argmax but changes confidence and loss semantics.

Train and evaluate the real feature

Data, normalization policy and confidence decide whether accuracy is useful.

Split by source, not random near-duplicate crops

Training data updates weights; validation chooses settings; calibration collects quantization ranges; test estimates final quality. Keep repeated photos, document pages and capture sessions from crossing the split. Define exact dictionary order, Unicode normalization, supported scripts and label length. For multi-code-point scripts, specify what one CTC token means before computing repeats or CER.

Metric / signal	Why you need it
Exact string accuracy	One wrong letter fails the whole result. Declare case, spaces and punctuation rules.
CER	(substitutions + deletions + insertions) / reference symbols; reveals the type and extent of damage.
Confidence + coverage	Measure correctness among accepted strings and how many inputs are accepted.
Alignment health	Track all-blank outputs, impossible targets and repeated-character deletions.

A confidence score needs a calibration test

PaddleOCR's CTC decoder averages selected path probabilities after duplicate and blank removal. This is a useful score, not a calibrated probability that the whole string is correct. It can hide a weak letter and ignores the sum over alternative valid CTC paths. Fit your acceptance policy on held-out data, with a false-accept target and separate script/length slices.

Fine-tuning and augmentation

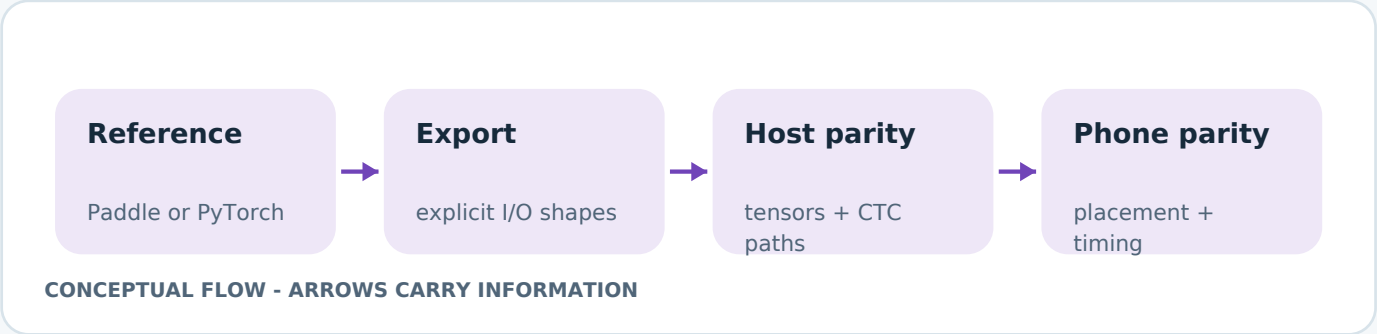
First overfit a tiny clean set to validate the pipeline, then return to a proper split. Use blur, noise, lighting and geometric perturbations representative of your camera. Never keep a full label when augmentation cropped off a required character. Match the paper's guidance phase only when intentionally reproducing it; otherwise document your fine-tuning recipe.

Worked error diagnosis

Reference BOOK, prediction BOK: exact accuracy is 0% and symbol-level CER is $1/4 = 25\%$. Inspect raw paths for a missing blank between O peaks, then check width capacity, crop detail and training repeats. CER alone cannot tell whether the cause is collapse logic or the network.

Export every supported shape

A simple CTC output path still needs graph and preprocessing parity.



Choose a route that matches the source implementation

Use a Paddle-compatible export path or validated conversion for PaddleOCR. OpenOCR's PyTorch reference is a candidate for ONNX or LiteRT Torch; the latter requires torch.export compatibility. Check support for attention, rectification sampling and dynamic shapes in the actual exported graph.

Start with a named static bucket

Export one exact input size and batch. Match resize, colour order, normalization, padding and layout to the reference. Compare features, scores, argmax IDs and decoded text. Add other MSR buckets deliberately; wide padding changes the image distribution and can affect blanks.

Execution target	What to prove
CPU	Correct operators, sensible thread settings and a reproducible baseline.
GPU / NPU	Operation coverage, placement, dtype/shape support, copies and synchronization cost.
Mixed partitions	Unsupported ops do not cause expensive repeated accelerator/CPU transfers.

Current Android constraint

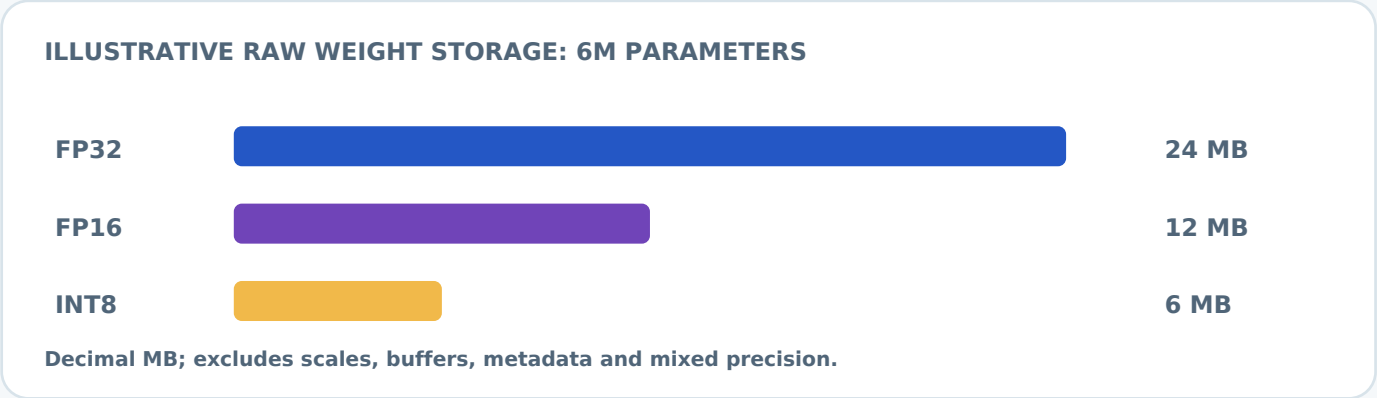
NNAPI was deprecated in Android 15. Check the actual SoC, SDK, driver and runtime backend. Snapdragon and Exynos paths may differ. Inspect placement traces and keep a tested fallback.

Integration is part of the benchmark

Reuse sessions/buffers, keep inference off the UI thread and bound the frame queue. Test resume, cancellation, low memory and sustained use. Verify that guidance branches are absent.

Quantize without losing the word

Numeric drift can change blanks, repeats and rare symbols.



The numerical approximation

Linear INT8 quantization reconstructs a value as $\hat{x} = \text{scale} \times (q - \text{zero_point})$. With $\text{scale}=0.02$, $\text{zero_point}=0$, $x=0.37$ rounds to $q=19$ and reconstructs as 0.38 . Errors accumulate differently across convolution, attention, normalization and the head. File-size reduction does not prove a corresponding compute speedup.

Method	What you validate
FP16	Supported kernels, score stability, blank/nonblank argmax and decoded parity.
PTQ	Representative calibration crops across sizes, scripts, contrast and long/repeated text.
QAT	Training simulates quantization effects; validate the final exported kernels and held-out quality.

CTC-specific acceptance checks

Compare FP32 and candidate scores on paired inputs. Look for newly deleted repeats, blank domination, rare symbol substitutions and confidence shifts. Tiny per-step errors near a decision boundary can remove an O or introduce a blank. Test mixed precision for sensitive attention/normalization/head operations where supported, using end-to-end strings as the acceptance criterion.

Do not quantize a mask as ordinary data

Original local attention adds -infinity to blocked scores. Preserve the masking semantics through export and quantization. A finite substitute that is not sufficiently negative can leak attention; an all-blocked row can yield NaNs. Calibrate the actual inference graph, including FRM if present.

TRY 10 Can one changed blank decision turn BOOK into BOK?

Answer >

Profile width, crops and heat

CTC avoids token feedback, but the visual model still has scaling costs.

TOY ATTENTION SCORE COUNTS - one head

200 tokens		40,000
400 tokens		160,000

2x token count -> 4x dense pairwise scores, not necessarily 4x total latency.

Scaling intuition with explicit assumptions

If height and channel width stay fixed, doubling image width doubles the spatial token count. Dense global attention pairwise work grows roughly with N^2 . Convolutional work, projection work and classifier work have other scaling terms. In v2, FRM also mixes row-width sequences. Measure each input bucket; do not extrapolate an entire feature from one short word.

An end-to-end toy budget

Suppose a serial crop pipeline spends 8 ms preparing input, 20 ms in the network and 2 ms decoding. Total is **30 ms**. Halving network time gives 20 ms total: **1.5x** speedup. The values are hypothetical. Add detection, capture, queueing and transfers when timing the full OCR product; dozens of crops can multiply recognition work.

Record the measurements that matter

Measure cold load/compile time, warm p50/p95 latency, crops per second, peak RAM and sustained energy per useful result. Synchronize asynchronous backends before stopping timers. Record device/SoC, OS, runtime, precision, input bucket, batch, thermal state and camera workload. GPU paper timings are not flagship phone guarantees.

Optimization experiments in a useful order

Remove copies and session reloads; group compatible crops; test a smaller variant; then precision and graph fusion. Distillation trains a student from teacher outputs/features. Structured pruning removes channels/layers that the backend can exploit; scattered zeros do not necessarily speed dense kernels. Validate new errors and sustained behavior after every change.

Debug it, then check your answers

Keep one observable hypothesis for each failure.

Failure	First observation	Next test
BOOK -> BOK	Raw IDs around both O characters	Check collapse order, blank separation and minimum T.
Long labels disappear	Sequence length after combining / FRM	Check out_char_num, width buckets and target repeat counts.
Only device results differ	Exact input tensor and first divergent layer	Check colour/layout, resize, mask, precision and output softmax.

CHECKPOINT ANSWERS - tap the number to return

- 01** No. Blank emits nothing; a supported visible space is an ordinary symbol.
- 02** Four. The two stride-2 stages multiply: $2 \times 2 = 4$.
- 03** No. The inspected original code forms dense scores before adding the mask.
- 04** No. [2,1] halves height while preserving width.
- 05** OO. First merge O O to O, giving O _ O; then remove blank.
- 06** Five: four labels plus one extra blank slot for the adjacent OO pair.
- 07** No. Backbone, neck, transform and head define the full architecture.
- 08** 160. $\text{floor}(5.9) \times 32 = 5 \times 32$; target size is 32×160 .
- 09** No. SGM uses labels during training and is omitted in inference.
- 10** Yes. A missing blank between equal O peaks can merge the two Os.

Last three distinctions

A CTC sequence position is not a known final character. SVTR block-type ordering is not PARSeq language permutation. SVTRv2's FRM stays at inference while SGM does not. These distinctions determine what to inspect in code and what to include in a device profile.

Your first SVTR engineering lab

Make each result reproducible by a teammate.

- ☐ **Baseline:** Run original SVTR or named v2 weights on 20 diverse crops.
- ☐ **CTC:** Verify repeats, blank removal, dictionary IDs and minimum T.
- ☐ **Variants:** Compare two sizes on paired crops; separate v1 from v2.
- ☐ **Export:** Prove tensor/string parity for every deployed shape bucket.
- ☐ **Phone:** Measure cold, warm and sustained latency, memory and errors.

Completion target: pinned config/checkpoint/dictionary, saved regression results, verified export and a device trace with explicit conditions. Agree accuracy, latency and memory budgets for the actual feature.

PRIMARY SOURCES - every row is clickable

- P1** SVTR original paper: IJCAI 2022
- P2** SVTRv2 paper + supplementary: ICCV 2025
- P3** PARSeq: architecture and permutation training
- T1** CTC: original alignment objective
- C1** PaddleOCR: SVTRNet stem, mixers and merging
- C2** PaddleOCR: full SVTR configuration
- C3** PaddleOCR: SVTR neck implementation
- C4** PaddleOCR: CTC head and eval softmax
- C5** PaddleOCR: decoding, repeats and confidence
- C6** PaddleOCR: recognizer image transforms
- V1** OpenOCR: SVTRv2 two-convolution encoder
- V2** OpenOCR: feature rearrangement / RCTC decoder
- V3** OpenOCR: Tiny, Small, Base and guidance configs
- V4** OpenOCR: SVTRv2 training/inference instructions
- V5** OpenOCR: CTC loss and zero_infinity
- M1** ONNX Runtime: mobile deployment and partitioning
- M2** ONNX Runtime: quantization and drift debugging
- M3** LiteRT: PyTorch conversion requirements
- M4** Android: NNAPI deprecation and migration

Official code/configs and runtime sources checked on 8 October 2026. Pin commits in your own experiments. Diagrams and numeric examples are original teaching illustrations except explicitly attributed architecture specifications. This guide does not claim Samsung internals, a tested export or measured phone latency.